

Mass loss model for pebble aggregate samples in DiMES

Erick Martinez Loran

August 2026

1 The mass loss is proportional to the incident heat load

$$\frac{dm}{dt} = c q_{\text{incident}} = c q_{\parallel} A_w, \quad (1)$$

where $q_{\parallel}$ is the parallel heat load at the divertor and A_w is the wetted area: $A_w(t) = Dh(t)/2$, with $h(t)$ the height of the rod exposed to the plasma at time t .

Consider the incident heat loss as

$$q_{\parallel} = \gamma n_e k_B T_e c_s, \quad (2)$$

where γ is the transmission coefficient in the sheath, k_B is Boltzmann's constant, c_s is the ion sound speed, T_e , and n_e are the electron temperature and electron density at the sheath edge, respectively.

The ion saturation current density J_{sat} is the maximum current that can be drawn to a material surface (such as a Langmuir probe or vacuum chamber wall) and is defined as follows:

$$J_{\text{sat}} \equiv e n_{\text{se}} c_s \quad (3)$$

Substitute $n_{\text{se}} c_s = J_{\text{sat}}/e$ into the sheath transmission equation Eq. (2):

$$q_{\parallel} = \frac{\gamma J_{\text{sat}} k_B T_e}{e}. \quad (4)$$

Multiply Eq. (4) by the wetted area A_w and plug it into Eq. (1):

$$\frac{dm}{dt} = c A_w \frac{\gamma J_{\text{sat}} k_B T_e}{e} \quad (5)$$

We propose that $A_w(t) J_{\text{sat}}(t)$ is the current measured at DiMES:

$$\frac{dm}{dt} = c \gamma k_B I(t) T_e(t).$$

The prefactor $c \gamma k_B$ can be lumped into a single constant c

$$\boxed{\frac{dm}{dt} = c I(t) T_e(t)} \quad (6)$$

To calibrate the model, we measure the total mass change $m_L \equiv \Delta m = m_f - m_i$ before and after exposure to the plasma heat load. Then,

$$m_L = c \int_0^t I(t) T_e(t) dt = c Q. \quad (7)$$

So:

$$c = \frac{m_L}{Q} \quad (8)$$

2 Estimating the uncertainty in c

The total uncertainty is given by:

$$\delta c = \left[\left(\frac{\delta m_L}{m_L} \right)^2 + \left(\frac{\delta Q}{Q} \right)^2 \right]^{1/2} \quad (9)$$

2.1 Q is estimated numerically

Q is estimated using either the trapezoid or Simpson's rule.

Simpson's rule

$$Q = \frac{\Delta t}{3} \sum_{i=0}^N w_i I_i T_{e,i}, \quad w_i \in \{1, 4, 2, 4, 2, \dots, 4, 1\} \quad (10)$$

So each contribution is $q_i = \frac{\Delta t}{3} w_i I_i T_{e,i}$:

$$Q = \sum_{i=0}^N q_i$$

The partial derivatives of q_i are

$$\frac{\partial q_i}{\partial I_i} = \frac{q_i}{I_i}, \quad \frac{\partial q_i}{\partial T_{e,i}} = \frac{q_i}{T_{e,i}}$$

So

$$\begin{aligned} \delta q_i &= \left[\left(\frac{q_i \delta I_i^2}{I_i} \right)^2 + \left(\frac{q_i \delta T_{e,i}^2}{T_{e,i}} \right)^2 \right]^{1/2} \\ &= q_i \left[\left(\frac{\delta I_i^2}{I_i} \right)^2 + \left(\frac{\delta T_{e,i}^2}{T_{e,i}} \right)^2 \right]^{1/2} \end{aligned}$$

The uncertainty in Q is

$$\delta Q = \left[\sum_i \left(\frac{\partial Q}{\partial q_i} \right)^2 \delta q_i^2 \right]^{1/2} = \left[\sum_i \delta q_i^2 \right]^{1/2}$$

Then,

$$\delta Q = \left\{ \sum_i q_i^2 \left[\left(\frac{\delta I_i}{I_i} \right)^2 + \left(\frac{\delta T_{e,i}}{T_{e,i}} \right)^2 \right] \right\}^{1/2}$$

Expanding q_i :

$$\delta Q = \left\{ \sum_i \left(\frac{\Delta t}{3} w_i I_i T_{e,i} \right)^2 \left(\frac{\delta I_i}{I_i} \right)^2 + \sum_i \left(\frac{\Delta t}{3} w_i I_i T_{e,i} \right)^2 \left(\frac{\delta T_{e,i}}{T_{e,i}} \right)^2 \right\}^{1/2}$$

Considering the systematic uncertainty on the current: $\delta I_i / I_i \equiv \epsilon_I$ (from considering a $\epsilon_I = 5\%$ tolerance resistor and $I = V/R \Rightarrow \delta I = I \delta R / R = \epsilon_I I$), and canceling terms:

$$\delta Q = \left\{ \sum_i \left(\frac{\Delta t}{3} w_i T_{e,i} I_i \epsilon_I \right)^2 + \sum_i \left(\frac{\Delta t}{3} w_i I_i \delta T_{e,i} \right)^2 \right\}^{1/2}$$

The first term is equal to $\epsilon_I Q$:

$$\delta Q = \left\{ (\epsilon_I Q)^2 + \sum_i \left(\frac{\Delta t}{3} w_i I_i \delta T_{e,i} \right)^2 \right\}^{1/2} \quad (11)$$

This can be implemented in Python as follows:

Code 1: Example implementation of δQ in Python

```

1 import numpy as np
2 from scipy.integrate import simpson
3
4 # --- Inputs ---
5 # t, I, Te, dTe are 1D arrays of equal length N
6 # (N-1 must be even for pure Simpson)
7 # ddm = uncertainty in delta_m, dm = delta_m value
8 epsilon_I = 0.05 # delta_R / R
9
10 # --- Integral via Simpson's rule ---
11 integrand = I * Te
12 Q = simpson(integrand, x=t)
13
14 # --- Build Simpson weights explicitly ---
15 N = len(t)
16 dt = t[1] - t[0] # assumes uniform spacing
17
18 def simpson_weights(N, dt):
19     """Returns Simpson's 1/3 rule weights for N points (N must be odd)."""
20     assert N % 2 == 1, "Need odd number of points (even number of intervals)"
21     w = np.ones(N)
22     w[1:-1:2] = 4 # odd indices
23     w[2:-2:2] = 2 # even indices (interior)
24     return w * dt / 3
25
26 w = simpson_weights(N, dt)
27
28 # --- Uncertainty in Q ---
29 dQ_sys = epsilon_I * Q # systematic (R calibration)
30 dQ_stat = np.sqrt(np.sum((w * I * dTe)**2)) # statistical (Te errors)
31 dQ = np.sqrt(dQ_sys**2 + dQ_stat**2)
32
33 # --- Uncertainty in c ---
34 c = dm / Q
35 rel_unc_c = np.sqrt((ddm / dm)**2 + (dQ / Q)**2)
36 delta_c = c * rel_unc_c
37
38 print(f"Q = {Q:.4e}")
39 print(f"dQ (sys) = {dQ_sys:.4e} ({100*dQ_sys/Q:.1f}%)")
40 print(f"dQ (stat) = {dQ_stat:.4e} ({100*dQ_stat/Q:.1f}%)")
41 print(f"dQ total = {dQ:.4e} ({100*dQ/Q:.1f}%)")
42 print(f"c = {c:.4e}")
43 print(f"delta_c = {delta_c:.4e} ({100*rel_unc_c:.1f}%)")

```

3 Expanding the uncertainty in the rate of mass loss

The rate of mass loss is given by Eq. (6):

$$r(t) = \frac{dm}{dt} = cI(t)T_e(t). \quad (12)$$

The constant c , the current $I(t)$, and the electron temperature $T_e(t)$ have associated uncertainties δc , $\delta I = \epsilon_I$, and δT_e .

Writing Eq. (12) explicitly

$$r(t) = \frac{m_L}{\int_0^\tau I(t)T_e(t)dt} I(t)T_e(t).$$

Take the logarithm

$$\begin{aligned} \ln r(t) &= \ln(m_L) + \ln[I(t)] + \ln[T_e(t)] - \ln\left(\int_0^\tau I(t)T_e(t)dt\right) \\ &= \ln m_L + \ln I + \ln T_e - \ln Q \end{aligned} \quad (13)$$

Take the partial derivative

$$\frac{\partial \ln r}{\partial m_L} = \frac{1}{r} \frac{\partial r}{\partial m_L} = \frac{1}{m_L} \quad (14)$$

Varying each error source, **one at a time**:

3.1 Variation 1: δm_L (independent)

$$\left(\frac{\delta r}{r}\right)_{m_L} = \frac{1}{r} \left(\frac{\partial r}{\partial m_L}\right) \delta m_L = \frac{\delta m_L}{m_L}. \quad (8)$$

3.2 Variation 2: $\delta R/R$ (systematic, affects all I_i and $I(t)$)

Every I_i has propagates the uncertainty as $I_i \rightarrow I_i(1 + \epsilon)$ and $I(t) \rightarrow I(t)(1 + \epsilon)$.

We are assuming that the error is systematic: R has a fractional error $\epsilon = \delta R/R$.

Since $I = V/R$, a resistor that is **larger than assumed** means the **current is smaller than assumed**:

$$I_{\text{true}} = \frac{V}{R_{\text{true}}} = \frac{V}{R(1 + \epsilon)} \approx I(1 - \epsilon)$$

So strictly speaking $\delta I/I = -\epsilon$. However, for **uncertainty propagation, we only care about the magnitude** of the effect, not the sign. Writing $I \rightarrow I(1 + \epsilon)$, with $\epsilon = \pm \delta R/R$ is just a convenient way to track how a fractional shift propagates through the formula.

Then, $Q \rightarrow Q(1 + \epsilon)$:

$$Q = \int_0^\tau I(t)T_e(t)dt \rightarrow \int_0^\tau I(t)(1 + \epsilon)T_e(t)dt = (1 + \epsilon) \int_0^\tau I(t)T_e(t)dt = Q(1 + \epsilon).$$

This causes cancelation in r : Substitute both shifts in r :

$$\begin{aligned} r &= cI(t)T_e(t) = \frac{m_L}{Q} I(t)T_e(t) \\ r &\rightarrow \frac{m_L}{Q(1 + \epsilon)} I(t)(1 + \epsilon)T_e(t) = \frac{m_L}{Q} I(t)T_e(t) \frac{(1 + \epsilon)}{(1 + \epsilon)} = r \end{aligned}$$

The $(1 + \epsilon)$ factors cancel **exactly**, so r is completely insensitive to ϵ

3.3 Variation 3: δT_e

Assuming Simpson's rule, $Q = \frac{\Delta t}{3} \sum_i w_i I_i T_{e,i}$. Then,

$$\ln r_k = \ln m_L + \ln I_k + \ln T_{e,k} - \ln \left(\frac{\Delta t}{3} \sum_j w_j I_j T_{e,j} \right) \quad (15)$$

T_e appears **twice**:

- Once **explicitly** in $+\ln T_{e,k}$
- Once **implicitly** inside $-\ln Q$, at every time step j , including $j = k$

Case $j \neq k$: $T_{e,j}$ appears only inside $\ln Q$

$$\frac{\partial \ln r_k}{\partial T_{e,j}} = -\frac{\partial \ln Q}{\partial T_{e,j}} = -\frac{1}{Q} \frac{\partial Q}{\partial T_{e,j}} = -\frac{w_j I_j \frac{\Delta t}{3}}{Q} \quad (9)$$

Case $j = k$: $T_{e,j}$ appears in both terms $\ln T_{e,k}$, and $-\ln Q$ simultaneously

$$\begin{aligned} \frac{\partial \ln r_k}{\partial T_{e,k}} &= \underbrace{\frac{\partial \ln T_{e,k}}{\partial T_{e,k}}}_{\text{explicit term}} - \underbrace{\frac{\partial \ln Q}{\partial T_{e,k}}}_{\text{implicit term}} \\ &= \frac{1}{T_{e,k}} - \frac{w_j I_j \frac{\Delta t}{3}}{Q} \end{aligned} \quad (16)$$

Multiplying through by $\delta T_{e,k}$ to express as a relative uncertainty:

$$\frac{\partial \ln r_k}{\partial T_{e,k}} \delta T_{e,k} = \left[1 - \frac{w_k I_k T_{e,k} \frac{\Delta t}{3}}{Q} \right] \frac{\delta T_{e,k}}{T_{e,k}} \quad (17)$$

Define

$$\alpha_k \equiv \frac{w_k I_k T_{e,k} \frac{\Delta t}{3}}{Q} \quad (18)$$

3.4 Full T_e uncertainty

We can unify both cases cleanly

$$\left(\frac{\delta r_k}{r_k} \right)_{T_e}^2 = \sum_j \left(\frac{\partial \ln r_k}{\partial T_{e,j}} \delta T_{e,j} \right)^2 = \sum_j \left(\frac{\partial}{\partial T_{e,j}} [\ln T_{e,k} - \ln Q] \delta T_{e,j} \right)^2 \quad (19)$$

where

$$\frac{\partial \ln r_k}{\partial T_{e,j}} = \begin{cases} \frac{1 - \alpha_k}{T_{e,k}}, & j = k \\ -\frac{w_j I_j \frac{\Delta t}{3}}{Q}, & j \neq k \end{cases} \quad (20)$$

So the sum over **all** j naturally splits into the two cases:

$$\left(\frac{\delta r_k}{r_k} \right)^2 = (1 - \alpha_k)^2 \left(\frac{\delta T_{e,k}}{T_{e,k}} \right)^2 + \sum_{j \neq k} \left(\frac{w_j I_j \frac{\Delta t}{3}}{Q} \right)^2 \delta T_{e,j}^2 \quad (21)$$

This can be easily implemented in **Python**

Code 2: Sample implementation of δr_k in Python

```

1 import numpy as np
2
3 # --- Trapezoid weights ---
4 def trapezoid_weights(t):
5     """Non-uniform spacing supported."""
6     dt = np.diff(t)
7     w = np.zeros(len(t))
8     w[:-1] += dt / 2
9     w[1:] += dt / 2
10    return w
11
12 w = trapezoid_weights(t)
13
14 # --- Integral ---
15 Q = np.sum(w * I * Te)
16
17 # --- Alpha: fractional contribution of each time step ---
18 alpha = w * I * Te / Q # shape (N,), sums to 1
19
20 # --- Uncertainty in r(t) ---
21 r = c * I * Te
22
23 # Term 1: delta_m (scalar)
24 frac_dm = (ddm / dm)**2
25
26 # Terms 2+3: T_e contributions, split at each t
27 # For each index k, the local term is (1 - alpha[k])**2 * (dTe[k]/Te[k])**2
28 # and the sum over j != k of (w[j]*I[j]*dTe[j]/Q)**2
29
30 full_Te_sum = np.sum((w * I * dTe / Q)**2) # sum over ALL j
31 local_Te     = (w * I * dTe / Q)**2        # contribution of each j
32                                           # to sum
33
34 # For index k:
35 #   sum_{j != k} = full_Te_sum - local_Te[k]
36 #   local term   = (1 - alpha[k])**2 * (dTe[k]/Te[k])**2
37
38 frac_Te_nonlocal = full_Te_sum - local_Te # array, sum over j!=t
39 frac_Te_local    = (1 - alpha)**2 * (dTe / Te)**2 # array, local t term
40
41 delta_r_over_r = np.sqrt(frac_dm + frac_Te_local + frac_Te_nonlocal)
42 delta_r = r * delta_r_over_r

```

4 Propagating uncertainty in $m_{\text{lost}}(t)$

The cumulative eroded mass up to time step k is:

$$m_k = cS_k = m_L \left(\frac{S_k}{S} \right), \quad (22)$$

where $S_k \equiv \sum_{i=0}^k w_i I_i T_{e,i}$, and $S \equiv \sum_{i=0}^N w_i I_i T_{e,i} = Q/(\Delta t/3)$. For simplicity, we designate m as the emitted (or lost) mass m_{lost} .

Taking the log:

$$\ln m_k = \ln m_L + \ln S_k - \ln S \quad (23)$$

4.1 Variations

4.1.1 δm_L (independent)

$$\frac{\partial \ln m_k}{\partial m_L} = \frac{1}{m_L} \implies \left(\frac{\delta m_k}{m_k} \right)_{m_L} = \frac{\delta m_L}{m_L} \quad (24)$$

4.1.2 $\delta R/R$ (systematic, all I_i scale together)

Every $I_i \rightarrow I_i(1 + \epsilon)$, so $S_k \rightarrow S_k(1 + \epsilon)$, and $S \rightarrow S(1 + \epsilon)$.

$$\begin{aligned} \delta \ln m_k &= \ln [S_k(1 + \epsilon)] - \ln [S(1 + \epsilon)] - \ln S_k + \ln S \\ &= \ln(1 + \epsilon) - \ln(1 + \epsilon) = 0 \end{aligned}$$

Cancels exactly, same argument as for r_k .

4.1.3 $\delta T_{e,j}$ – Three cases now

Since $\ln m_k = \ln m_L + \ln S_k - \ln S$, varying $T_{e,j}$, gives

$$\frac{\partial \ln m_k}{\partial T_{e,j}} = \frac{\partial \ln S_k}{\partial T_{e,j}} - \frac{\partial \ln S}{\partial T_{e,j}} \quad (25)$$

The key difference from r_k : there is **no explicit $\ln T_{e,k}$ term**. T_e only appears inside the two S_k and S . *Why do we need to vary over all values of j ?* Because m_k depends on **all** $T_{e,j}$ values, not just the local one at $j = k$: Look at $m_k = m_L (S_k/S) = m_L \left(\sum_{i=0}^k w_i I_i T_{e,j} \right) / \left(\sum_{i=0}^N w_i I_i T_{e,j} \right)$. Every single $T_{e,j}$ for $j = 0, \dots, N$ appears somewhere in this expression.

Case 1: $j > k$ – appears only in S , not in S_k

$$\begin{aligned} \frac{\partial \ln m_k}{\partial T_{e,j}} &= \frac{1}{S_k} \frac{\partial S_k}{\partial T_{e,j}} - \frac{1}{S} \frac{\partial S}{\partial T_{e,j}} \\ &= \frac{1}{S_k} \sum_{i=0}^k \underbrace{\frac{\partial}{\partial T_{e,j}} (w_i I_i T_{e,i})}_{=0 \text{ since } j > k} - \frac{1}{S} \sum_{i=0}^N \underbrace{\frac{\partial}{\partial T_{e,j}} (w_i I_i T_{e,i})}_{=\delta_{ij}} \\ &= -\frac{w_j I_j}{S} \\ \implies \frac{\partial \ln m_k}{\partial T_{e,j}} \delta T_{e,j} &= -\frac{w_j I_j}{S} \delta T_{e,j} \end{aligned}$$

Case 2: $j < k$ – appears in both S_k , and S

$$\begin{aligned} \frac{\partial \ln m_k}{\partial T_{e,j}} &= \frac{1}{S_k} \sum_{i=0}^k \underbrace{\frac{\partial}{\partial T_{e,j}} (w_i I_i T_{e,i})}_{=\delta_{ij}} - \frac{1}{S} \sum_{i=0}^N \underbrace{\frac{\partial}{\partial T_{e,j}} (w_i I_i T_{e,i})}_{=\delta_{ij}} \\ &= \frac{w_j I_j}{S_k} - \frac{w_j I_j}{S} = w_j I_j \left(\frac{1}{S_k} - \frac{1}{S} \right) \\ \implies \frac{\partial \ln m_k}{\partial T_{e,j}} \delta T_{e,j} &= w_j I_j \left(\frac{1}{S_k} - \frac{1}{S} \right) \delta T_{e,j} \end{aligned}$$

Case 3: $j = k$ – also appears in both S_k , and S

$$\begin{aligned}\frac{\partial \ln m_k}{\partial T_{e,k}} &= \frac{1}{S_k} \sum_{i=0}^k \underbrace{\frac{\partial}{\partial T_{e,k}} (w_i I_i T_{e,i})}_{=\delta_{ik}} - \frac{1}{S} \sum_{i=0}^N \underbrace{\frac{\partial}{\partial T_{e,k}} (w_i I_i T_{e,i})}_{=\delta_{ik}} \\ &= \frac{w_k I_k}{S_k} - \frac{w_k I_k}{S} = w_k I_k \left(\frac{1}{S_k} - \frac{1}{S} \right) \\ \Rightarrow \frac{\partial \ln m_k}{\partial T_{e,k}} \delta T_{e,k} &= w_k I_k \left(\frac{1}{S_k} - \frac{1}{S} \right) \delta T_{e,k}\end{aligned}$$

The full expression for $\frac{\partial \ln m_k}{\partial T_{e,j}} \delta T_{e,j}$ is

$$\frac{\partial \ln m_k}{\partial T_{e,k}} \delta T_{e,j} = \begin{cases} w_j I_j \left(\frac{1}{S_k} - \frac{1}{S} \right) \delta T_{e,j} & j \leq k \\ -\frac{w_j I_j}{S} \delta T_{e,j} & j > k \end{cases} \quad (26)$$

4.1.4 Full result

$$\frac{\delta m_k}{m_k} = \left\{ \left(\frac{\delta m_L}{m_L} \right)^2 + \sum_{j \leq k} \left[w_j I_j \left(\frac{1}{S_k} - \frac{1}{S} \right) \delta T_{e,j} \right]^2 + \sum_{j > k} \left[\frac{w_j I_j}{S} \delta T_{e,j} \right]^2 \right\}^{1/2} \quad (27)$$

This can be easily implemented in **Python**:

Code 3: Example code to estimate δm_k in Python

```
1 # S_k: cumulative sum up to each index k
2 S_k = np.cumsum(w * I * Te) # shape (N,)
3 S    = S_k[-1]              # full sum = Q
4
5 # Precompute per-step weight * current
6 wI = w * I                  # shape (N,)
7
8 # For each k, build delta_m_k/m_k
9 frac_dm = (ddm / dm)**2     # scalar
10
11 frac_Te = np.zeros(N)
12 for k in range(N):
13     past = np.arange(0, k+1) # j <= k
14     future = np.arange(k+1, N) # j > k
15     term_past = np.sum((wI[past] * (1/S_k[k] - 1/S) * dTe[past])**2)
16     term_future = np.sum((wI[future] * dTe[future] / S)**2)
17     frac_Te[k] = term_past + term_future
18
19 delta_m_over_m = np.sqrt(frac_dm + frac_Te)
```

5 Change of A_w with incident heat load

We assume that the wetted area A_w decreases with exposure to heat load. This should be proportional to the mass loss:

$$A_w \equiv Dh(t)/2$$

where D is the diameter of the rod and $h(t)$ is the height of the rod exposed to the plasma at time t_k . $h(t)$ decreases with mass loss:

$$\Delta h(t) \propto m_{\text{lost}}(t)$$

with m_{lost} as defined in Eq. (22). We assume D remains constant.

The volume of the rod lost during exposure is $V(t) = \pi D^2 h(t)/4$. Assuming a constant effective density ρ , $m_{\text{lost}}(t)/\rho = \pi D^2 \Delta h(t)/4$. Then,

$$\Delta h(t) = \frac{4}{\rho \pi D^2} m_{\text{lost}}(t)$$

And the wetted area is given by

$$A_w(t) = \frac{D}{2} \left(\frac{4}{\rho_{\text{rod}} \pi D^2} \right) m_{\text{lost}}(t) = \frac{2}{\rho_{\text{rod}} \pi D} m_{\text{lost}}(t) \quad (28)$$

5.1 Uncertainty propagation in A_w

The quantities ρ_{rod} , D , and m_{lost} have associated uncertainties $\delta \rho_{\text{rod}}$, δD , and δm_{lost} . So the uncertainty in A_w is given by

$$\delta A_w(t) = A_w(t) \left\{ \left(\frac{\delta D}{D} \right)^2 + \left(\frac{\delta \rho_{\text{rod}}}{\rho_{\text{rod}}} \right)^2 + \left(\frac{\delta m_{\text{lost}}}{m_{\text{lost}}} \right)^2 \right\}^{1/2} \quad (29)$$

5.2 Surface recession rate

The surface recession rate for the wetted surface

$$v \equiv \frac{1}{\rho_{\text{rod}} A_w(t)} \frac{dm(t)}{dt} \quad (30)$$

Plugging in Eq. (28):

$$v(t) = \frac{1}{\rho_{\text{rod}}} \left(\frac{\rho_{\text{rod}} \pi D}{2 m_{\text{lost}}(t)} \right) r(t) = \frac{\pi D}{2} \frac{r(t)}{m_{\text{lost}}(t)}$$

Which can be written **explicitly** in terms of $I(t)$, and $T_e(t)$ as

$$v(t) = \frac{\pi D}{2} \frac{1}{c \int_0^t I(t') T_e(t') dt} c I(t) T_e(t) \quad (31)$$

Expressing it in numeric form:

$$v_k = \frac{\pi D}{2} \frac{I_k T_{e,k}}{S_k} \quad (32)$$

5.3 Uncertainty in surface recession rate

Start with the expression in Eq. (32):

$$v_k = \frac{\pi D}{2} \frac{I_k T_{e,k}}{S_k}.$$

Define

$$X_i \equiv I_i T_{e,i}, \quad S_k \equiv \sum_{i=0}^k w_i X_i \quad (33)$$

The propagation of the uncertainty of $f(u_1, \dots, u_n)$ is given by

$$\sigma_f^2 = \sum_m \left(\frac{\partial f}{\partial u_m} \right)^2 \sigma_m^2$$

Assume $\delta V = 0$, so that the error in $I = V/R$ is systematic from $\delta R/R = \epsilon$. Because R is constant across measurements, **even with a nonzero ϵ , it does not contribute to δ_k at all.**

Here is why, if $I_j = V_j/R_j$ with the *same* R is used for every sample, then substitute it directly into ν_k before ever differentiating:

$$\nu_k = \frac{\pi D}{2} \frac{I_k T_{e,k}}{S_k} = \frac{\pi D}{2} \frac{(V_k/R) T_{e,k}}{\sum_i^k w_i (V_i/R) T_{e,i}} = \frac{\pi D}{2} \frac{V_k T_{e,k}}{\sum_i^k w_i V_i T_{e,i}}$$

R factors out of every term in the sum S_k and out of the numerator identically, so it cancels completely –algebraically, before taking any derivative.

5.3.1 The D partial

$\nu_k \propto D$ exactly, so

$$\frac{\partial \nu_k}{\partial D} = \frac{\nu_k}{D} \quad (34)$$

trivial.

5.3.2 The X_j partial

Rather than differentiating directly with respect to $T_{e,j}$, it is cleaner to first get $\partial \nu_k / \partial X_j$, then use the chain rule through $X_j = I_j T_{e,j}$.

Write $\nu_k = \frac{\pi D}{2} X_k S_k^{-1}$ and use the product rule

$$\frac{\partial \nu_k}{\partial X_j} = \frac{\pi D}{2} \left[\frac{\partial X_k}{\partial X_j} \cdot \frac{1}{S_k} + X_k \cdot \frac{\partial (S_k)^{-1}}{\partial X_j} \right]$$

Two pieces to work out:

- $\frac{\partial X_k}{\partial X_j} = \delta_{jk}$ (the Kronecker delta – X_k only depends on itself, so this is 1 if $j = k$, 0 otherwise). **This is the term that captures the numerator/denominator correlation** – X_k appears explicitly in the numerator, so differentiating the numerator with respect to X_j only “sees” it when $j = k$.
- $\frac{\partial S_k}{\partial X_j} = w_j$ for $j \leq k$ (since $S_k = \sum_{i=0}^k w_i X_i$ is linear in each X_i), so by the chain rule, $\frac{\partial (S_k)^{-1}}{\partial X_j} = -S_k^{-2} \frac{\partial S_k}{\partial X_j} = -\frac{w_j}{S_k^2}$

Putting these together:

$$\frac{\partial \nu_k}{\partial X_j} = \frac{\pi D}{2} \left[\frac{\delta_{jk}}{S_k} - \frac{w_j X_k}{S_k^2} \right], \quad j \leq k \quad (35)$$

5.3.3 Chain rule down to $T_{e,j}$

Since $X_j = I_j T_{e,j}$, with no error through I_j (systematic error $\delta R/R$ cancels out and $\delta V_j = 0$ assumed),

$$\frac{\partial v_k}{\partial T_{e,j}} = \frac{\partial v_k}{\partial X_j} \cdot \frac{\partial X_j}{\partial T_{e,j}} = \frac{\partial v_k}{\partial X_j} \cdot I_j$$

so in variance form

$$\frac{\partial v_k}{\partial T_{e,j}} \delta T_{e,j} = \frac{\pi D}{2} \left[\frac{\delta_{jk}}{S_k} - \frac{w_j X_k}{S_k^2} \right] I_j \delta T_{e,j}, \quad j \leq k$$

It is cleanest to just define $\delta X_j \equiv I_j \delta T_{e,j}$ and treat $\left(\frac{\partial v_k}{\partial X_j} \right)^2 \delta X_j^2$ as the contribution.

5.3.4 Assemble the total variance

Sum the independent contributions from D and each $T_{e,j}$, $j = 0, 1, \dots, k$:

$$\delta v_k = \left\{ \left(\frac{v_k}{D} \right)^2 \delta D^2 + \sum_{j=0}^k \left[\frac{\pi D}{2} \left(\frac{\delta_{jk}}{S_k} - \frac{w_j X_k}{S_k^2} \right) \right]^2 \delta X_j^2 \right\}^{1/2} \quad (36)$$

This can be implemented in Python as

Code 4: Example implementation of the recession rate calculation in Python

```

1  """
2  Uncertainty propagation for the DiMES surface recession rate
3
4      nu_k = (pi * D / 2) * X_k / S_k,  X_i = I_i * Te_i,  S_k = sum_{i=0}^k w_i * X_i
5      I_i  = V_i / R    (same R for every sample; I treated as exact -- see below)
6
7  Notes
8  -----
9  - rho and c cancel analytically and do not appear as inputs.
10 - R cancels identically out of nu_k (homogeneous degree 0 in the I_i's), so a
11   systematic calibration error dR/R contributes nothing and is not modeled.
12 - Voltage/current uncertainty is taken as zero (machine epsilon), so I_i is
13   treated as exact. The only per-sample randomness is in Te_i.
14 - dX_i therefore reduces to X_i * (dTe_i / Te_i) = I_i * dTe_i.
15 """
16
17 import numpy as np
18
19
20 def propagate_analytic(D, dD, I, Te, dTe, w):
21     """
22     Parameters
23     -----
24     D, dD : float
25         Rod diameter and its 1-sigma uncertainty.
26     I : array_like, shape (N,)
27         Current samples (I = V/R), treated as exact (dV = 0).
28     Te, dTe : array_like, shape (N,)
29         Electron temperature samples and their 1-sigma uncertainties.
30     w : array_like, shape (N,)

```

```

31         Quadrature weights (e.g. trapezoidal dt's). Assumed exact.
32
33     Returns
34     -----
35     nu, dnu : ndarray, shape (N,)
36     """
37     I, Te, dTe, w = (np.asarray(a, dtype=float) for a in (I, Te, dTe, w))
38     N = len(I)
39
40     X = I * Te
41     dX = I * dTe # = X * (dTe/Te), since dI/I = 0
42
43     S = np.cumsum(w * X)
44     nu = (np.pi * D / 2.0) * X / S
45
46     dnu = np.empty(N)
47     for k in range(N):
48         Sk = S[k]
49         j = np.arange(k + 1)
50         kronecker = (j == k).astype(float)
51         dnu_dXj = (np.pi * D / 2.0) * (kronecker / Sk - w[j] * X[k] / Sk ** 2)
52         var_X = np.sum((dnu_dXj * dX[j]) ** 2)
53         var_D = (nu[k] / D * dD) ** 2
54         dnu[k] = np.sqrt(var_X + var_D)
55
56     return nu, dnu
57
58
59 def propagate_montecarlo(D, dD, I, Te, dTe, w, n_draws=20000, seed=0):
60     """Monte Carlo cross-check (I fixed/exact, only D and Te resampled)."""
61     rng = np.random.default_rng(seed)
62     I, Te, dTe, w = (np.asarray(a, dtype=float) for a in (I, Te, dTe, w))
63     N = len(I)
64
65     D_s = rng.normal(D, dD, size=n_draws)
66     Te_s = rng.normal(Te, dTe, size=(n_draws, N))
67
68     X_s = I[None, :] * Te_s
69     S_s = np.cumsum(w[None, :] * X_s, axis=1)
70     nu_s = (np.pi * D_s[:, None] / 2.0) * X_s / S_s
71
72     return nu_s.mean(axis=0), nu_s.std(axis=0)
73
74
75 if __name__ == "__main__":
76     N = 50
77     t = np.linspace(0, 1, N)
78     I = 5.0 + 3.0 * np.sin(np.pi * t) # placeholder current trace (exact)
79     Te = 20.0 + 10.0 * t
80     dTe = 0.10 * Te
81     w = np.gradient(t)
82     D, dD = 0.6, 0.02
83
84     nu, dnu = propagate_analytic(D, dD, I, Te, dTe, w)
85     nu_mc, dnu_mc = propagate_montecarlo(D, dD, I, Te, dTe, w)
86
87     print(f"{'k':>3} {'nu':>10} {'dnu (analytic)':>16} {'dnu (MC)':>10}")

```

```
88     for k in (0, N // 4, N // 2, 3 * N // 4, N - 1):
89         print(f"{k:3d} {nu[k]:10.4f} {dnu[k]:16.4f} {dnu_mc[k]:10.4f}")
```